

HTTP/2 y HTTP/3

Redes y Comunicaciones (2026)

¿Qué es HTTP/2?

- Nueva forma de transportar HTTP sobre una conexión.
- No cambia los métodos, códigos de estado ni la semántica de HTTP.
- Su diseño tomó como antecedente al protocolo SPDY desarrollado por Google.
- Definido en:
 - RFC 9113: HTTP/2 (reemplaza al RFC 7540).
 - RFC 7541: HPACK – Header Compression for HTTP/2.

Problemas con HTTP/1.0, HTTP/1.1

- No intercala varias respuestas en una conexión al mismo tiempo.
- Alternativas usadas para reducir el bloqueo (HOL):
 - Conexiones persistentes y pipelining.
 - Generar conexiones paralelas.
- Problemas:
 - Pipelining, requiere respuestas en el orden, puede generar HOL.
 - Clientes evitan aplicar pipelining a POST (no idempotente).
 - Demasiadas conexiones: usa muchos recursos de SO, control de congestión y uso ineficiente de la red.
 - Muchos requests repiten gran cantidad de encabezados.

Diferencias principales entre HTTP2 y HTTP/1.1

- Framing binario en lugar de mensajes ASCII delimitados como texto.
- Una conexión TCP: varios streams, streams divididos en frames.
- Una conexión para pedir/traer datos en paralelo: datos fuera de orden, priorización, flow control por frame.
- Server Push parte del protocolo, pero en desuso por los navegadores, no lograba mejoras.
- Usa compresión de encabezados con HPACK.
- El estándar permite HTTP/2 sin TLS, pero los navegadores lo usan normalmente sobre HTTPS y lo negocian mediante ALPN.

Server Push: característica en desuso

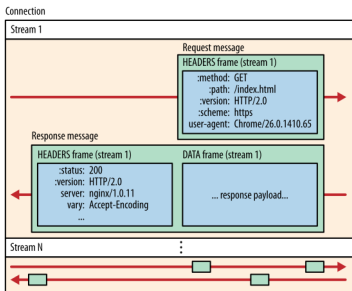
- El servidor podía anticipar recursos mediante `PUSH_PROMISE` y enviar la respuesta sin esperar un request explícito.
- En la práctica no se logra una mejoría, estado de caché del cliente desconocida por el server, mandaba info duplicada.
- Configuración era compleja y los beneficios medidos fueron inconsistentes; Chrome/Chromium eliminó su soporte desde la versión 106.
- Alternativas actuales:
 - `<link rel="preload">` o encabezado `HTTP Link: rel=preload.`
 - Respuesta informativa `103 Early Hints`, adelanta esos enlaces.

HTTP/2 mux stream, framing

- HTTP/2 intenta reutilizar una conexión TCP por origen para enviar los mensajes.
- Puede usar la misma conexión por diferentes sites, mismo destino IP/TLS: (coalescing).
- Conexión TCP usada por HTTP2 se divide/mux en streams (simulan “sub-conexiones”).
- Un stream lleva un mensaje HTTP2 a la vez (intercambio request/response).
- Stream: es un flujo lógico bidireccional con ID dentro de la conexión TCP que lleva el mensaje HTTP2.
- Stream IDs: 32bits, diferentes por clientes (impares) vs. por servidor (pares), se agotan.
- Una vez usado el stream este se cierra, no se reutiliza.
- GOAWAY, vaciado y nueva conexión.

HTTP/2 mux stream, framing (cont.)

- Frames de distintos streams se pueden intercalar en la misma conexión.
- Los mensajes están compuestos por frames de diferentes tipos.
- Frame types: HEADERS, DATA, PUSH_PROMISE, WINDOW_UPDATE, SETTINGS, etc.



fuelle: <https://web.dev/performance-http2/#streams,-messages,-and-frames>

HTTP/2 mux stream, framing (cont.)

http2_native.pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http2

No.	Time	Source	Destination	Protocol	Length	Info
4	0.414129282	192.168.0.228	139.162.123.134	HTTP2	90	Magic
5	0.420956333	192.168.0.228	139.162.123.134	HTTP2	152	SETTINGS[0], WINDOW_UPDATE[0], HEADERS[1]: G
7	0.839101976	139.162.123.134	192.168.0.228	HTTP2	100	SETTINGS[0]
10	0.839139220	139.162.123.134	192.168.0.228	HTTP2	76	SETTINGS[0]
12	0.839167233	139.162.123.134	192.168.0.228	HTTP2	369	HEADERS[1]: 200 OK, DATA[1] (text/plain)
14	0.839280935	192.168.0.228	139.162.123.134	HTTP2	75	SETTINGS[0]
15	0.845040952	192.168.0.228	139.162.123.134	HTTP2	93	HEADERS[3]: GET /humans.txt

Transmission Control Protocol, Src Port: 38564, Dst Port: 80, Seq: 25, Ack: 1, Len: 86

HyperText Transfer Protocol 2

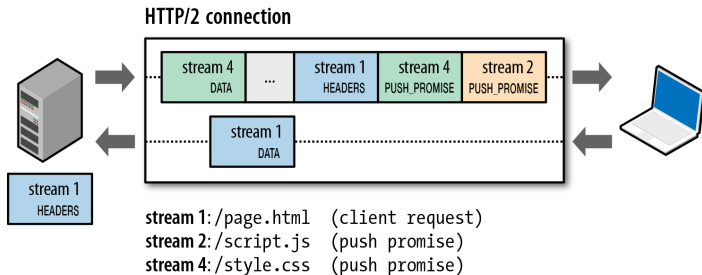
- Stream: SETTINGS, Stream ID: 0, Length 18
- Stream: WINDOW_UPDATE, Stream ID: 0, Length 4
- Stream: HEADERS, Stream ID: 1, Length 37, GET /robots.txt
 - Length: 37
 - Type: HEADERS (1)
 - Flags: 0x05
 - 0... .. = Reserved: 0x0
 - .000 0000 0000 0000 0000 0000 0000 0001 = Stream Identifier: 1
 - [Pad Length: 0]
 - Header Block Fragment: 82048862c3c674a174f94f864188aa69d29ac4b9ec9b7a88...
 - [Header Length: 136]
 - [Header Count: 6]
 - Header: :method: GET
 - Header: :path: /robots.txt
 - Header: :scheme: http
 - Header: :authority: nghttp2.org
 - Header: user-agent: curl/7.61.0
 - Header: accept: */*

HyperText Transfer Protocol 2: Protocol

Packets: 21 · Displayed: 8 (38.1%)

Profile: Classic

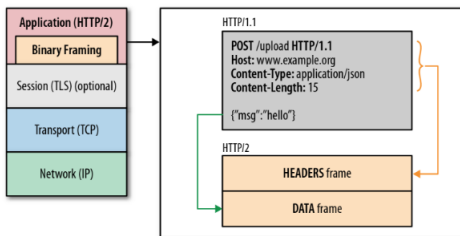
HTTP/2 mux stream, framing (cont.)



fuelle: <https://docs.google.com/presentation/d/1r7QXGYOLCh4fcUq0jDdDwKJWNqWK1o4xMtYpKZCJYjM/present?slide=id.p19>

HTTP/2 mux stream, framing (Cont.)

- Mensaje en el stream dividido en frames, frame = header fijo + payload variable.
- Los frames se codifican en binario y tienen un header común fijo de 9 bytes.



fuelle: <https://docs.google.com/presentation/d/1r7QXGYOLCh4fcUq0jDdDwKJWNqWK1o4xMtYpKZCJYjM/present?slide=id.p19>

HTTP/2 HEADERS

- Se mantienen casi todos los HEADERS de HTTP/1.1.
- No se usa texto ASCII: HPACK codifica los campos.
- Surgen pseudo-headers que contienen información que estaba en la línea inicial y otros headers.
- Por ejemplo: `HEAD /algo HTTP/1.1` se reemplaza con `http2:`
 - `:method: head`
 - `:path: /algo`
 - `:scheme: https` o `http`
 - `:authority: www.site.com` reemplaza al `headerHost:`.

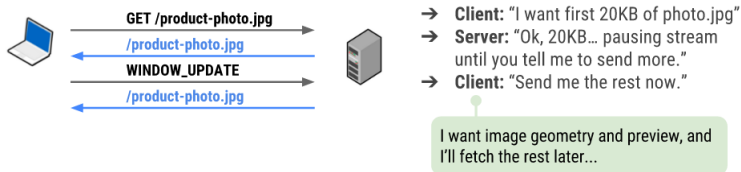
Para las respuestas: `:status:` contiene códigos de estado como 200, 301 o 404.

HTTP/2: Priorización de Stream

- RFC 7540 definía: prioridades `weight : 1-256`, relación jerárquica, `priority-frames`.
- Este esquema original de pesos y dependencias del RFC 7540 está obsoleto.
- Esquema original de árbol de jerarquía resultó complejo de implementar.
- RFC 9218 define prioridades extensibles: `urgency`, `incremental`; sugerencias para el servidor, no garantizadas.

HTTP/2: Control de flujo

- Control de flujo por conexión por TCP (ventana de TCP).
- Control de flujo por stream: capacidad de Rx (ventana del stream).
- El receptor anuncia capacidad con frames:
`SETTINGS_INITIAL_WINDOW_SIZE` y actualiza con frames:
`WINDOW_UPDATE`.
- Sólo los frames `DATA` consumen la ventana de HTTP/2 del stream.

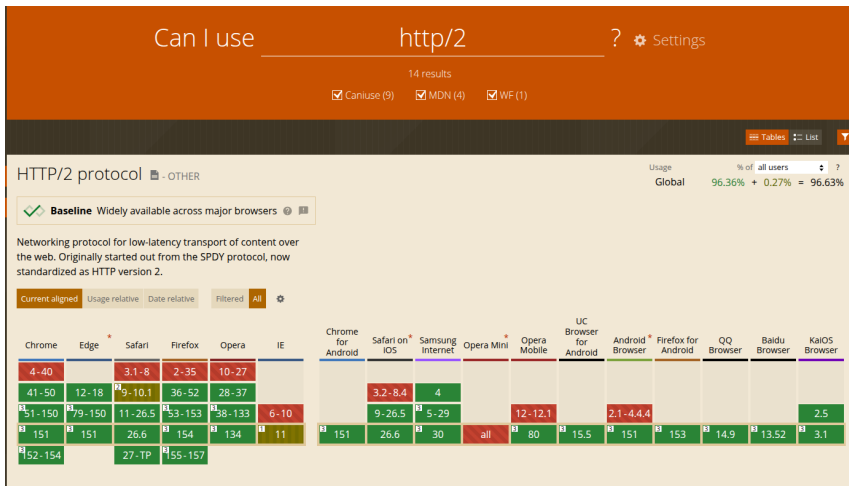


fuelle: <https://docs.google.com/presentation/d/1r7QXGYOLCh4fcUq0jDdDwKJWNqWK1o4xMtYpKZCJYjM/present?slide=id.p19>

Compresión y soporte

- HTTP/2 comprime encabezados para evitar repetir campos entre requests.
- Las primeras versiones de SPDY/2 utilizaron compresión basada en DEFLATE (gzip).
- La compresión con contexto y datos secretos habilitó ataques como CRIME.
- HTTP/2 usa HPACK, diseñado para reducir esos riesgos y el tamaño de los encabezados.
- HTTP/2 tiene soporte amplio; SPDY obsoleto.

Soporte en clientes para 2026



fuelle: <http://caniuse.com/#search=HTTP%2F2>

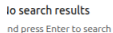
Inicio y negociación de HTTP/2

- RFC 7540 permitía `HTTP2-Settings:` y `Upgrade: h2 | h2c`, RFC 9113 lo deja obsoleto.
- Para HTTP/2 se negocia durante TLS mediante ALPN: se ofrecen las versiones soportadas: ej: `h2 | http/1.1`; el servidor selecciona uno.
- Para HTTP/2 sin TLS sólo queda el modo *prior knowledge*: cliente asume que el servidor es h2c (casi no se implementa).
- HTTP/3 suele descubrirse mediante `Alt-Svc:` o registros DNS HTTPS/SVCB (RFC 9460).
`a.com. 3600 IN HTTPS 1 svc.a.net. alpn='h3,h2'`
- Dentro de la conexión QUIC, ALPN selecciona h3.

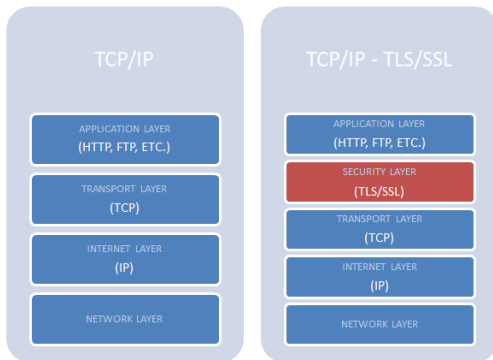
Application-Layer Protocol Neg.

- Captura ClientHello TLS 1.3 con la extensión ALPN.
- El cliente ofrece h2 y http/1.1; el servidor selecciona uno de la lista.

tls					
No.	Time	Source	Destination	Protocol	Length Info
					Type: server_name (0)
					Length: 20
					▸ Server Name Indication extension
					▾ Extension: extended_master_secret (len=0)
					Type: extended_master_secret (23)
					Length: 0
					▾ Extension: renegotiation_info (len=1)
					Type: renegotiation_info (65281)
					Length: 1
					▸ Renegotiation Info extension
					▸ Extension: supported_groups (len=10)
					▸ Extension: ec_point_formats (len=2)
					▸ Extension: session_ticket (len=0)
					▾ Extension: application_layer_protocol_negotiation (len=14)
					Type: application_layer_protocol_negotiation (16)
					Length: 14
					ALPN Extension Length: 12
					▾ ALPN Protocol
					ALPN string length: 2
					ALPN Next Protocol: h2
					ALPN string length: 8
					ALPN Next Protocol: http/1.1
					▸ Extension: status_request (len=5)
					▸ Extension: signature_algorithms (len=10)
					▸ Extension: signed_certificate_timestamp (len=0)



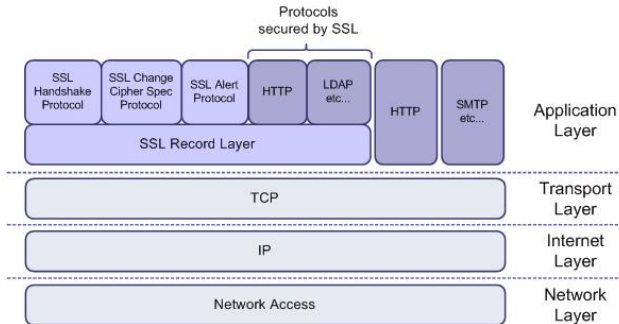
SSL/TLS



Network Layer = Network Access Layer = Network Interface Layer = Link + Física

fuelle: <https://www.simple-talk.com/dotnet/net-framework/tlsssl-and-net-framework-4-0/>

SSL/TLS (Cont.)

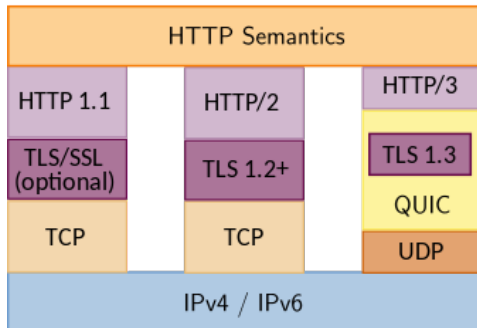


fuelle: http://nicolascormier.com/documentation/bin/apache/apache2_with_ssl_tls/part1.htm

¿Qué es HTTP/3?

- Transporta HTTP sobre QUIC, normalmente sobre UDP puerto 443.
- Conserva los métodos, códigos y semántica común de HTTP.
- QUIC se originó en Google y luego fue estandarizado por la IETF.
- Definido en:
 - RFC9000: QUIC: A UDP-Based Multiplexed and Secure Transport.
 - RFC9114: HTTP/3.

HTTP/3



fuelle: <https://en.wikipedia.org/wiki/HTTP/3>

Diferencias principales de HTTP/3-QUIC

- QUIC integra el establecimiento del transporte y TLS 1.3: normalmente requiere 1 RTT en una primera conexión y puede usar 0-RTT al reanudar.
- Evita el head-of-line blocking de TCP entre streams: la pérdida en un stream no detiene a los demás. Dentro de un mismo stream todavía hay orden.
- Implementa recuperación de pérdidas y control de congestión sobre UDP; no elimina esos controles.
- Las retransmisiones usan nuevos números de paquete; esto mejora la detección de pérdidas y evita la ambigüedad de retransmisión de TCP.
- Casi todos los paquetes QUIC están autenticados y cifrados; TLS 1.3 está integrado al protocolo.
- Los Connection IDs permiten mantener la conexión ante ciertos cambios de IP o puerto, facilitando la migración.

Diferencias principales de HTTP/3-QUIC (Cont.)

- Algunos middleboxes o redes bloquean o degradan UDP; el cliente puede volver a HTTP/2 sobre TCP.
- QUIC suele implementarse en espacio de usuario, lo que facilita evolucionarlo sin actualizar el kernel.
- Al usar UDP debe limitar ataques de amplificación; QUIC incorpora validación de dirección y límites de amplificación.

HTTP/3 captura

*enp0s31f6

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

quic

No.	Time	Source	Destination	Protocol
21	9.202350845		2800:3f0:4002:809::2003	QUIC
22	9.202542084		2800:3f0:4002:809::2003	QUIC

Internet Protocol Version 6, Src: 2800:340: Dst: 2800:3f0:4002:809::2003

User Datagram Protocol, Src Port: 56546, Dst Port: 443

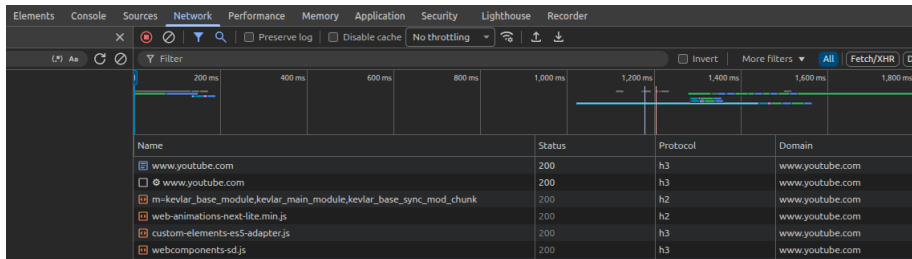
QUIC IETF

- QUIC Connection information
 - [Packet Length: 1230]
 - 1... = Header Form: Long Header (1)
 - .1... = Fixed Bit: True
 - ..00 = Packet Type: Initial (0)
 - ... 00.. = Reserved: 0
 -00 = Packet Number Length: 1 bytes (0)
 - Version: 1 (0x00000001)
 - Destination Connection ID Length: 8
 - Destination Connection ID: 5b8c72711860d7e2
 - Source Connection ID Length: 0
 - Token Length: 70
 - Token: 00a724a2bbb3734560b9583c50a85edabdd33ef2a8f75a68ae4ec82ff9041010b80dbdf3...
 - Length: 1141
 - Packet Number: 1
 - Payload: d7b50a500c2fdf8454cd5916cc5100508a1778eae56de71f0604326db821547415afac4d...
- PING
 - Frame Type: PING (0x0000000000000001)
- PING
- PING
- PING
- PADDING Length: 18
- CRYPTO
 - Frame Type: CRYPTO (0x0000000000000006)
 - Offset: 312
 - Length: 746
 - Crypto Data
 - TLV1.3 Record Layer: Handshake Protocol: Encrypted Handshake Message
- PING
- CRYPTO

Destination Connection ID Length (quic.dci), 1 byte

Packets: 2015 · Displayed: 670 (33.3%) Profile: Classic

HTTP/3 Debugging (2026)



[HTTP/2] <https://http2.github.io/>.

[RFC 9113] <https://www.rfc-editor.org/rfc/rfc9113.html>

[RFC 9218] <https://www.rfc-editor.org/rfc/rfc9218.html>

[RFC 9114] <https://www.rfc-editor.org/rfc/rfc9114.html>

[RFC 8297] <https://www.rfc-editor.org/rfc/rfc8297.html>

[Chrome – Server Push] <https://developer.chrome.com/blog/removing-push>

[Ilya Grigorik] [HTTP/2 is here, let's optimize!](https://docs.google.com/presentation/d/1r7QXGYOLCh4fcUq0jDdDwKJWNqWK1o4xMtYpKZCJYjM/present?slide=id.p19)

<https://docs.google.com/presentation/d/1r7QXGYOLCh4fcUq0jDdDwKJWNqWK1o4xMtYpKZCJYjM/present?slide=id.p19>.

[HTTP/2-dev] <https://web.dev/performance-http2/#streams,-messages,-and-frames>

[HTTP/3-cloudflare] <https://www.cloudflare.com/learning/performance/what-is-http3/>

[HTTP/3-wikipedia] <https://en.wikipedia.org/wiki/HTTP/3>

[QUIC-wikipedia] <https://en.wikipedia.org/wiki/QUIC>

[QUIC-google] <https://peering.google.com/#/learn-more/quic>